

Weighted Sum, Bias, Nonlinearity: ML Models and the Anatomy of a Neural Network

Lecture 3 essay — Outline

The Core Idea

- **Abstraction:** treat a component as a black box and attend only to how to use it (the car engine, the operating system). So far the training process was the box; this lecture opens a different one — **the model itself** — and the next opens training
- Three movements: (1) the landscape — the paradigms a model can learn under, and what separates a neural network from every earlier model; (2) a field guide to which model fits which data; (3) the anatomy of the simplest network — neurons, weights, biases, activations — and the one formula connecting them
- The test of understanding: explain why a neuron trained to detect the crossbar of a 7 fires on a 5 but not on a 6

I. Three Ways to Learn

- Three nested terms: **AI** (any technique that mimics human intelligence) $\supset$ **machine learning** (learns from data) $\supset$ **deep learning** (learns from data with a neural network). Cutting across all three: the **learning paradigm**, the structure of the problem itself. Any paradigm can be “deep”
- **Supervised learning:** predict y from x . The useful aspects of x are **features**; y is the **label**, the ground truth. Having the label is what makes it the easiest paradigm — a clean reference signal at every step. Discrete y (which city?) $\rightarrow$ **classification**; continuous y (what price?) $\rightarrow$ **regression**. The default for projects in this program
- **Unsupervised learning:** drop the label. **Clustering** (k -means) finds the clouds in the data without being told they exist — the first thing to try on unstructured data with suspected hidden regularity, with the caveat that unsupervised learning is hard. **Dimensionality reduction** (PCA, t-SNE) projects high-dimensional data down while keeping the main patterns — watching a video at 144p. It works because real data *lives on a low-dimensional manifold* (a wavy blanket in 3-space that flattens onto a plane). One-hot tokens ($\approx 50,000$ dimensions) $\rightarrow$ embeddings (a few thousand) is this idea. Rule of thumb: nothing above three dimensions can be visualized (Hinton: picture three and say “fourteen” very loudly)
- **Reinforcement learning:** an **agent** acts in an **environment**; each **action** moves it to a new **state** and collects **reward**; it learns a **policy** — which action in which state — to maximize reward. The paradigm of agentic problems: robotics, games (AlphaGo), trading. For this course: post-training — **RLHF** (helpful, honest, harmless) and **RLVR** (verifiable reward for math and logic; the engine behind recent reasoning gains). Structural point: the reward can be anything and need not be differentiable, unlike the standard training loop. Caveats: unstable and expensive — do not attempt without prior training intuition. **Epsilon-greedy** = one rule for the exploration/exploitation tension (try many restaurants before settling)

II. What Makes a Neural Network Different

- LLMs are neural networks; not all neural networks are language models. **Architectures** = recurring structural patterns that work: the transformer is dominant, the convolutional network is the other you will meet regularly
- A neural network is a complicated function — a large collection of weights trained toward a task. Three differences from classical ML: **automatic feature extraction** (raw pixels or text go in, and the features

are discovered as a by-product of driving the loss down — nobody told the model that *king* and *queen* differ by gender; contrast human **feature engineering**: square footage, zip code, pool); **scalability** (keeps improving with data, parameters, compute); **expressiveness** (complex nonlinear relationships; the Universal Approximation Theorem stands behind this, out of scope)

III. A Brief Review

- The key insight of ML: show the computer examples rather than tell it the steps. **Weights** = the adjustable knobs; the training loop improves them from data; frozen weights are a function from input to output, and running it is **inference**
- Any model is a curve; training draws the line of best fit. Fit the pattern, not the noise: connecting every dot is **overfitting**. Defense: a **training** set (weights change) and a **validation** set (performance measured) — unseen data is the only honest estimate. Training loss falling while validation loss rises means memorization has begun
- Why ML took off: the transformer scales. GPT-2 → GPT-3 was not an architectural idea — bigger model, more data, more compute — and double descent meant scaling kept helping instead of overfitting

IV. Choosing a Model by Data Modality

- Order of questions: **modality** → task → trade-offs. What follows is exposure, not derivation
- **Text**: the answer is an LLM — general reasoners, natively multimodal. The other surviving family is **BERT**: understands but does not generate; its output is an embedding. Two uses: **classification** (ten thousand movie reviews, positive or negative) and **retrieval** (Google Search since 2019; what powers **RAG**, retrieval-augmented generation). The barrier is low: Hugging Face plus a coding agent gives a working classifier in fifteen minutes —

```
from transformers import pipeline
classifier = pipeline("sentiment-analysis")      # a fine-tuned BERT-family model
classifier("The pacing dragged but the ending landed.")
```

The recurring advice: the fastest way to build intuition for a model is to use it

- **Visual**: one architecture per task. Classification → the **convolutional neural network** (ResNet 2015, VGG): convolution layers as a long feature-extracting preprocessing stage, then an ordinary network. Object detection → YOLO. Segmentation (every pixel assigned to an object) → SAM, Meta's Segment Anything Model, now third generation; U-Nets out of fashion. Generation → GANs essentially dead, replaced by **diffusion** (start from noise, repeatedly denoise); not autoregressive — hence a small camp bullish on diffusion *language* models. **VLM** = by now just a modern multimodal LLM; **VLA** extends the transformer to actions (robotics). In every case use a **pretrained** model and **fine-tune**: from scratch costs tens of thousands and performs worse
- **Tabular**: a spreadsheet — rows are data points, columns are features. An MLP works; for time series, LSTMs or classical ARIMA. But this is where traditional ML earns its keep: linear regression (interpretable, which matters when a quant model starts losing money), decision trees, random forests, gradient-boosted trees such as XGBoost — still winning Kaggle tabular competitions outright. Text and images are too high-dimensional for these; on tables, do not discount them

V. Anatomy of a Neural Network

- Running example (3Blue1Brown): a handwritten digit goes in; the network says which of ten digits it is. Classification, on a toy task, small enough to see through
- **The high-level goal**: early neurons respond to tiny stroke fragments; the next layer combines fragments into parts (a loop at the top, a vertical line down the right); the final layer combines parts into digits (a

loop over a line is a 9). *Features grow larger and more complete layer by layer*; everything else is machinery for making that happen

- **Input and output:** the input layer is the flattened image, one neuron per pixel (brightness 0–255, read row by row). The output layer has one neuron per class. Same shape as the LLM output — a score per vocabulary entry: **next-token prediction is classification**
- **Neurons and activations:** each hidden neuron should come to represent a feature; its **activation** is a number measuring how present that feature is in the current input. A crossbar-of-7 neuron: high when the bar is there, low when it is not
- **Weights** live on edges — one per pair of neurons in adjacent layers; magnitude = how much this neuron cares about that one. Every neuron connects to every neuron in the next layer: dense, **fully connected**. (Course-wide, “weights” means every tunable number, biases included; inside a network the word narrows)
- **Biases** live on neurons — one per neuron in each **hidden layer**. Two arguments: $y = mx + b$, where the bias is the $+b$, an additive offset that adds flexibility and empirically improves generalization; and the biological one — a neuron fires only when $mx + b > \text{threshold}$, i.e. $mx > \text{threshold} - b$, so the bias sets how hard the neuron is to turn on. Both point the same way; biases are used because they work
- **The activation function** introduces nonlinearity. The sigmoid (squashes to 0–1) is the classic and is almost never used now. The modern default is **ReLU**, which is an if statement:

$$\text{ReLU}(x) = \max(0, x) = \begin{cases} x & x > 0 \\ 0 & x \leq 0. \end{cases}$$

Two stays two; negative five becomes zero. GELU and Leaky ReLU are what most people actually run. Why bother: strip the nonlinearities out and the whole network is a weighted sum of weighted sums — another weighted sum, *linear regression with extra steps*. The nonlinearity lets the network represent what a line cannot

- Every weight and bias starts random and is learned by gradient descent — the next lecture. For today: these are the knobs, and something turns them

VI. The Forward Pass

- Why this structure: to *communicate the useful information from one layer to the next*. The weights dial how much each neuron listens to each neuron before it; the bias dials how hard it is to switch on. Enough layers with the right dials and hierarchical features emerge
- The rule of the vanilla network — an **MLP**, multilayer perceptron — in English: *the activation of a neuron is the weighted sum, over every neuron in the previous layer, of that neuron’s activation times the connecting weight; plus a bias; wrapped in the activation function*. The intuition is a weighted average: each neuron decides how much to care about each neuron behind it. The loop-of-a-9 neuron puts large weights on its five stroke neurons and near-zero weights on everything else. Training is dialing those weights to those values
- **In math**, for one neuron i in layer L , summing over the neurons j of layer $L - 1$:

$$z_i^{(L)} = \sum_j w_{ij}^{(L)} a_j^{(L-1)} + b_i^{(L)}, \quad a_i^{(L)} = \sigma(z_i^{(L)}).$$

$a_j^{(L-1)}$ is the j th previous activation, $w_{ij}^{(L)}$ the weight on the edge from j to i , $b_i^{(L)}$ the current neuron’s bias; z is the **pre-activation** and σ the activation function. A textbook compresses the whole layer into matrix algebra, the same statement at once for every neuron:

$$\mathbf{a}^{(L)} = \sigma(W^{(L)}\mathbf{a}^{(L-1)} + \mathbf{b}^{(L)}).$$

It looks worse than it is; ten minutes of staring resolves it into the English sentence above

- **Conceptual check:** a third-layer neuron has learned the horizontal crossbar of a 7. Pass in a 6 $\rightarrow$ **low** (no such bar). Pass in a 5 whose top stroke sits where the crossbar would $\rightarrow$ **high** (the feature is present, though the digit differs). Why, from the formula: split the previous layer into group A (the crossbar region, weights ≈ 1) and group B (everywhere else, weights ≈ 0). On the 6, group A gives 1×0 and group B gives $0 \times \text{anything}$, so the sum is ≈ 0 . On the 5, some group-A activations become 1, the 1×1 products add up to something substantial, and group B still contributes nothing. *The weights select a region, the activations report what is in it, and the sum says whether the feature is present*

VII. Caveats on the Idealized Picture

- One clean feature per neuron, composing hierarchically, is what we *hope* a trained network does. First complication: a model can reach low validation loss with *no interpretable features at all*. Adversarial robustness lives on this gap — pure noise classified as a 9 at 95% confidence. Frontier models turn out fairly interpretable when studied carefully, but training does not guarantee it
- Second: **polysemanticity**. A single neuron in a modern transformer often responds to many unrelated things at once — the loop of a 9, dogs, the letter A. The current untangling technique is the **sparse autoencoder**, which decomposes polysemantic activations into a sparser one-feature-per-direction representation (Anthropic’s interpretability publications; a mentor’s recording in the Drive)

VIII. From the Q&A

- *Which activation function?* ReLU or GELU; almost nothing else. Reason previews the next lecture: training multiplies many numbers, and small products vanish while large ones explode — the **vanishing and exploding gradient** problems. ReLU-family functions are better behaved than the sigmoid, however well-motivated the sigmoid is in theory
- *What determines the weights?* Nothing you set: random initialization, then the training loop
- *Does importance determine activation?* No. Activation is only how present the feature is; importance is a separate question (pruning research reaches the same performance with far fewer neurons)
- *A short 7 versus a tall 7?* The crossbar lands on different pixels, so different input neurons see it; a plain MLP copes imperfectly. This is the motivation for the **CNN**: convolving and pooling over neighborhoods learns patterns that survive small translations
- *Why not set the weights to the minimum directly?* You cannot see the loss surface — millions of dimensions, no map. Finding the maximum of a 100,000-element array by inspection is not possible; you follow the slope (tomorrow’s topic)
- *Deliberately overfit first?* Yes — it is in Karpathy’s training recipe — because it confirms the pipeline can learn at all before you pull back

Where to Go From Here

- Watch the 3Blue1Brown video; it covers the second half a different way. Attention remains a black box: the recommended series takes hours and is optional — benchmarking, prompting, and decoding projects can treat the model as a box, interpretability and efficiency projects cannot
- Use the models: running BERT, a segmentation model, or a diffusion model is now a natural-language request to a coding agent, and fifteen minutes of trying beats an hour of watching. Do not master every architecture horizontally; pick a project and go deep. Research ideas begin with auditions at the start of week 3 and lock in by week 5 or 6
- One sentence: a neural network is a stack of layers in which every neuron computes a weighted sum of the layer before it, adds a bias, and applies a nonlinearity — and, trained well, that repeated operation turns pixels into strokes, strokes into loops, and loops into a 9

Quick review

What’s strong: The crossbar conceptual check is the essay’s load-bearing device. It turns the formula into a two-group arithmetic argument (group A weights 1, group B weights 0) that a reader can run in their head, and it is placed immediately after the notation so the symbols are grounded before they can intimidate. Distinguishing the course-wide sense of “weights” (every tunable number) from the in-network sense heads off a confusion that otherwise lasts weeks. The argument for nonlinearity — a weighted sum of weighted sums is a weighted sum — is pitched at exactly the right level: no theorem, just algebra the reader already owns. The modality field guide is honestly labelled as exposure rather than derivation, and “use a pretrained model” is repeated at every point where a student might be tempted not to.

Where to push: (1) Section II lists expressiveness as a third way networks differ from classical models, but expressiveness is a consequence of the nonlinearity introduced in Section V; a forward pointer would let the reader meet the same fact twice and recognize it. (2) The bias discussion offers two stories and says both “point the same way,” but the threshold story is a rename of the $y = mx + b$ story, not independent support; saying so is more honest than presenting two arguments. (3) Scalability is asserted without the caveat from lecture one that it holds in the double-descent regime and not before, so a reader could take it as unconditional. (4) The strokes-to-loops-to-digits picture is presented in V as what a trained network does and walked back only in VII; one clause of foreshadowing would prevent the reader from over-committing. (5) The RL section notes the reward “does not have to be differentiable” without saying what that buys — it is precisely why RL can optimize human preference rankings and pass/fail checkers, which is the entire reason RLHF and RLVR exist. (6) Minor: “fully connected” is introduced without its contrast — local or sparse connectivity, which is what a convolution is — so the CNN arrives in the Q&A tied to a question rather than to the anatomy.

Connection to your work: Automatic feature extraction is the fact that governance keeps tripping over. A network is handed raw data and decides for itself what to attend to, so “which features does the model use” is an empirical question answered after training, not a design specification anyone can be required to disclose in advance — which bounds what a documentation or transparency mandate can literally ask for. Section VII is the second hook: training does not guarantee interpretable features, and polysemanticity means a neuron is not a unit of meaning, which is the technical reason literal “explainability” requirements are hard to satisfy and why the sparse-autoencoder line of work matters for any audit regime. And the pretrained-then-fine-tune default in IV is the structural fact behind supply-chain questions: the party who fine-tunes is rarely the party who trained, so responsibility has two natural attachment points, not one.

The lecture material is from the Algoverse AI Research program, courtesy of Algoverse. The essay and this outline are the author’s own writing from those lectures, not copied from the lecture materials.